

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 701 — Cybersecurity

Defending systems you did not design against adversaries who only need one way in

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 9 (with CSCE 711 and CSCE 713)

Prerequisites: CSCE 611 Operating Systems and CSCE 678 Distributed Systems; CSCE 606 Software Engineering for the process material; CSCE 711 runs in parallel and supplies the cryptography

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: A threat model and security review of a system you operate or built, with the controls mapped to the threats, the residual risk stated, and a detection plan you have tested

Course description

This is a defensive course. It covers how systems fail so that you can build ones that do not, and **the governing asymmetry is that a defender must cover everything and an attacker needs one way in.** **Module 01 establishes that properly**, because every design decision in the course follows from it: defence in depth, least privilege, and assume-breach are all responses to the same arithmetic.

The second theme is that the interesting failures are almost never cryptographic. **CSCE 711 covers the primitives and they are, in practice, the part that works** — real incidents come from misconfiguration, credential reuse, missing authorisation checks, unpatched dependencies, and people. **So this course spends its time where the failures are**, and the cryptography chapter is a short one about key management (Module 07).

The third is that detection matters as much as prevention. **Assume a control will fail and ask how you would know** — which is Module 09, and it is the material most often missing from a security programme. **A system with perfect prevention and no logging cannot be investigated when the prevention turns out to have been imperfect.**

The fourth is that security is a property of a system in operation rather than of code. **Modules 10 and 11 cover incident response and human factors**, because **a control that people route around is not a control**, and an organisation that cannot respond to an incident has not prepared for the one it will have.

And the closing position is about honest claims. **Module 12 is about why security is hard to measure and which metrics actively mislead**, and **Module 13 is about what a security assessment can and cannot assert** — because 'we are secure' is not a statement anyone can support, and 'these threats are addressed by these controls, and these are accepted' is.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Build a threat model and defend its scope.
- 2 Explain authentication's real failure modes and design around them.
- 3 Design an authorisation scheme and reason about its weaknesses.
- 4 Explain what transport security and network segmentation do and do not provide.
- 5 Explain platform isolation mechanisms and their boundaries.
- 6 Explain the browser security model and the standard vulnerability classes.
- 7 Design key and secret management for a real system.
- 8 Assess and harden a software supply chain.
- 9 Design logging and detection that would catch a control failure.
- 10 Run an incident response exercise and write a blameless post-mortem.
- 11 Design controls that people will actually use.
- 12 Explain why security metrics mislead and choose better ones.
- 13 State what a security assessment honestly establishes.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	Thinking About Security	What makes securing a system structurally different from building one?
2	Authentication	How do you establish identity, and where does it go wrong?
3	Authorisation	Who is allowed to do what, and how do you get that consistently right?
4	Network Security	What does securing the network actually accomplish?
5	Platform and Isolation	What stops one program from reading another's data?
6	Web Security	What is the browser actually protecting, and from what?
7	Secrets and Keys	Where do the keys live, and who can use them?
8	The Supply Chain	Your code is 5% yours. What about the other 95%?
9	Detection	A control has failed. What tells you?
10	Incident Response	It has happened. What now?

Wk	Module	Core question it answers
11	Human Factors	Why do correct security designs fail in practice?
12	Measuring Security	How do you know whether you are more secure than last year?
13	Claiming Security Honestly	What can you actually say about a system's security?

Projects

Project 1 — A threat model, defended (after Module 7)

Threat-model a real system you built or operate, and map its controls to its threats.

Requirements

- **A system diagram** with trust boundaries marked explicitly.
- **Assets, adversaries, and their capabilities**, written out — not a generic list.
- **At least fifteen threats** identified by a systematic method, not by brainstorming.
- **Each threat mapped to a control, or explicitly accepted**, with the acceptance justified.
- **The authentication and authorisation design reviewed against Modules 02–03.**
- **Three threats you cannot address**, with the reason and what you would need.

Done means

- **The trust boundaries correct** — which is where most threat models go wrong, and it is graded.
- **The adversary model specific**: a bored stranger, a motivated competitor, and an insider are different threat models with different controls.
- **Every threat either mitigated or accepted in writing** — an unaddressed threat that is not acknowledged is the failure mode.
- **And the three unaddressable threats stated honestly**, which is the part that demonstrates judgement.

Project 2 — Detection, and a response exercise (after Module 12)

Build detection for the controls you identified, verify it fires, and run a response exercise.

Requirements

- **Logging designed for three specific threats** from Project 1, with the fields justified.
- **A detection rule per threat**, with the expected false positive rate estimated.
- **A test that triggers each detection** — on your own system, safely.
- **The base rate computed** (Module 09 §3) and the alert volume predicted.
- **A tabletop incident exercise**, with the timeline and the decisions recorded.
- **A blameless post-mortem** of the exercise, in Module 10's form.

Done means

- **Every detection actually tested and seen to fire** — an untested detection is a hope, and this is the central requirement.
- **The false positive rate estimated and the alert volume computed**, because an alert nobody reads is not a detection (Module 09 §3).

- **The exercise surfacing at least one gap you did not anticipate** — which it will.
- **And the post-mortem genuinely blameless**, identifying system causes rather than decisions a person should have made differently.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Anderson — Security Engineering, 3rd edition (free PDF)	The primary source, and free in full from the author. The best book on this subject by a wide margin, and it is about systems and incentives rather than about tools. Modules 01, 11, and 12 follow it closely.
Shostack — Threat Modeling: Designing for Security	Module 01's method. The four-question framework used throughout this course comes from here. Library copy; the author's free materials cover the core.
OWASP — the Application Security Verification Standard and the Cheat Sheet Series (free)	The practical reference for Modules 02–06 — specific, current, defensive, and maintained. The ASVS is the checklist to review a system against.
NIST SP 800-63 (digital identity) and SP 800-53 (controls) (free)	Module 02's authentication guidance, free — and notable for having reversed the industry's password advice on the basis of evidence, which is Module 11's point.
Google — Building Secure and Reliable Systems (free PDF)	Free in full, and the reference for Modules 05, 09, and 10 — security from the perspective of people operating large systems rather than auditing them.
Allspaw and the blameless post-mortem literature (free)	Module 10's practice. Short, and it changed how the industry handles incidents — which is a human-factors result rather than a technical one.

Tooling

- **A system you are authorised to examine. Your own machine, your own project, or a deliberately vulnerable training target** — and **nothing else, ever**, which is Module 01 §4's professional constraint and is not negotiable.
- **A deliberately vulnerable training environment:** OWASP Juice Shop, WebGoat, or a local Damn Vulnerable Web Application. **Free, legal, and designed for this.**
- **A dependency scanner** for Module 08 — pip-audit, npm audit, Dependabot, or Trivy. **Run it on your own projects and read the output.**
- **A log aggregator you can query**, even a local one, for Module 09. **Detection cannot be designed without somewhere to put the logs.**
- **A password manager and hardware security key**, used personally. **Module 02's material is far more convincing once you have lived with the controls**, including their friction.
- **And a written threat model template. The deliverable of this course is a document rather than code**, and the template is what makes it repeatable.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.